

NEW ERA UNIVERSITY
9 Central Ave, New Era, Quezon City, 1107 Metro Manila

Final Project in Data Structures and Algorithms

Al-amin B. Abdul
Jul-qarnain E. Cana
Vincent Zyrell Dela Cruz
Darryl Julian T. Montealegre
Nathan Samuel S. Orias
Russel Adrian E. Soterio
John Cedrix P. Villanueva

2BSIT-3

PROJECT SUMMARY

This project consists of three distinct programs, each designed to tackle a unique problem using Java. The first program contains the Simple Menu, it creates a menu system that allows the user to select from various options, with each option triggering specific operations through the use of switch statements. The second program, the Math Expression Calculator, focuses on processing fully parenthesized arithmetic expressions, converting them into binary expression trees. It displays the tree, calculates the value of the root, and supports variable leaves (x1, x2, x3) that can be updated interactively. Additionally, it provides the expression in both prefix and postfix notations. Last program three the Delivery Route Optimization Problem involves optimizing delivery routes in a small town with five districts, where each district contains multiple delivery points connected by various routes. The program aims to determine the most efficient route while considering varying package priorities. Each program demonstrates practical applications of Java programming techniques, including tree structures, interactive inputs, and optimization algorithms.

PROGRAM #1

CODE LINK

Write a Java program that presents a menu to the user and performs specific operations based on the selected option.

Data Structures used:

Structure 1: Array

- Used in the heightComparison method for storing names (String[] persons) and heights (int[] heights) of three individuals.

Algorithms Implemented:

Step 1: Start Program

- The program starts by initializing the Scanner object to read user inputs.

Step 2: User Input – Name

- The program asks the user to enter their name.

Step 3: Display Menu

- The program displays the menu options
- The user selects option 2 (Height Comparison).

Step 4: Height Comparison (Option 2)

- The program asks the user to input the names and heights of 3 persons
- Program Output: The program now compares the heights of the three persons

Step 5: Finding Tallest

- The method findingTallest is called with the names and heights of the three persons.
- The program then checks the value of tallestCtr

Step 6: Display Result

Step 7: Loop Continues

- After the height comparison, the program returns to the main menu.
- Prompt: The user is asked to choose another option.
- User Input: The user selects option 3 (Display Fibonacci Series).

Step 8: Display Fibonacci Series (Option 3)

- The program asks for the number of terms in the Fibonacci series.
- Program Output: The program calculates and displays the first terms of the Fibonacci series

Step 9: Loop Continues

- The program returns to the main menu again.
- Prompt: The user is asked to choose another option.
- The user selects option 4 (Display Diamond of Asterisks).

Step 10: Display Diamond of Asterisks (Option 4)

- The program asks for the number of rows in the diamond (the middle part).
- Program Output: The program displays a diamond-shaped pattern with rows

Step 11: Exit Program (Option 5)

- The user selects option 5 (Exit).
- The program terminates, closing the Scanner object.

Key Functions:

checkLeapYear(Scanner input)

- Checks if a given year is a leap year using arithmetic and logical operations.

printSeries(Scanner input)

- Generates and displays the first n terms of the Fibonacci series using iterative addition such as loop.

printDiamond(Scanner input)

- Prints a diamond pattern of asterisks based on the user-provided row count.

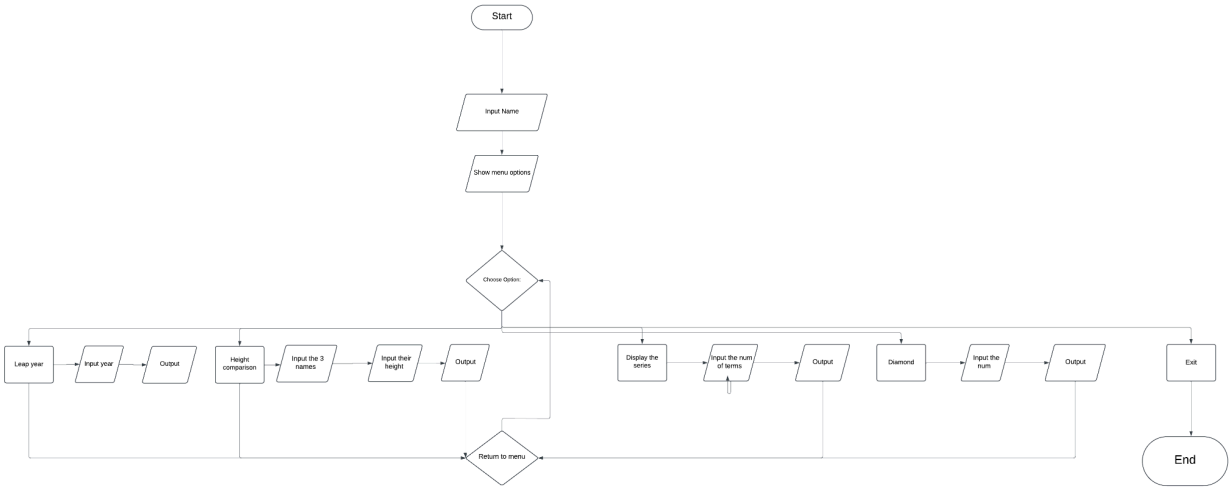
heightComparison(Scanner input)

- Gather names and heights of three individuals and determine the tallest person(s) by invoking findingTallest.

findingTallest(String[] persons, int[] heights)

- Determines the tallest height and returns a formatted string indicating the tallest individual(s).

Flowchart:



New Era University
COLLEGE OF INFORMATICS AND COMPUTING STUDIES
NO. 9 CENTRAL AVE., NEW ERA, QUEZON CITY, PHILIPPINES

PROGRAM #2

CODE LINK

Write a program that takes, as input, a fully parenthesized, arithmetic expression and converts it to a binary expression tree.

Data Structures used:

Structure 1: Binary Tree

- Used for an Expression Tree for arithmetic expressions. Each node represents either an operator (`+`, `-`, `*`, `/`) or an operand (integer or variable). Each node also contains a String value (operator or operand) and references to left and right child nodes.

Structure 2: HashMap

- Stores variables (keys) and their integer values, and also facilitates variable updates during the program execution.

Structure 3: Stacks

- Used to store intermediate tree nodes (operands and operator nodes during tree construction) and operators (`+`, `-`, `*`, `/`) while parsing the expression.

Algorithms Implemented:

Step 1: Start Program

- The program starts by creating an instance of Scanner to take user input.
- Enter a fully parenthesized arithmetic expression User Input: (A + B)
- The program processes the input and begins creating an expression tree.

Step 2: Build Expression Tree

- The program parses the user input (A + B) and creates an expression tree by iterating through each character of the string:
- The root node of the tree represents the + operator, with A and B as its left and right children, respectively.

Step 3: Display Expression Tree

- The program calls displayTree to print the expression tree in a horizontal layout.

Step 4: Display Prefix Notation

- The program generates the prefix notation for the expression tree.

Step 5: Display Postfix Notation

- The program generates the postfix notation for the expression tree.

Step 6: Evaluate the Expression

- The program evaluates the expression. Since no variables have been set, the evaluation will return an error (or a default result).

Step 7: Update Variable Values

- Prompt: Would you like to update variables? (Y/N): User Input: Y
- The user decides to update the variable values.
- The program updates the values for A and B to User Input: 5 and 3, respectively.

Step 8: Reevaluate the Expression

- The program reevaluates the expression with the updated values for A and B.
- The program evaluates the expression (5 + 3), which results in 8.

Step 9: Continue or Exit

- The user chooses to exit the program.

Key Functions:

buildTree(String expression)

- Constructs the binary expression tree from a fully parenthesized arithmetic expression.

evaluate(TreeNode node)

- Recursively evaluates the expression tree, returning the result as an integer.

displayTree(TreeNode node, int level)

- Displays the tree in a horizontal layout for visualization.

generatePrefix(TreeNode node, StringBuilder result)

- Generates and prints the prefix notation of the expression.

generatePostfix(TreeNode node, StringBuilder result)

- Generates and prints the postfix notation of the expression.

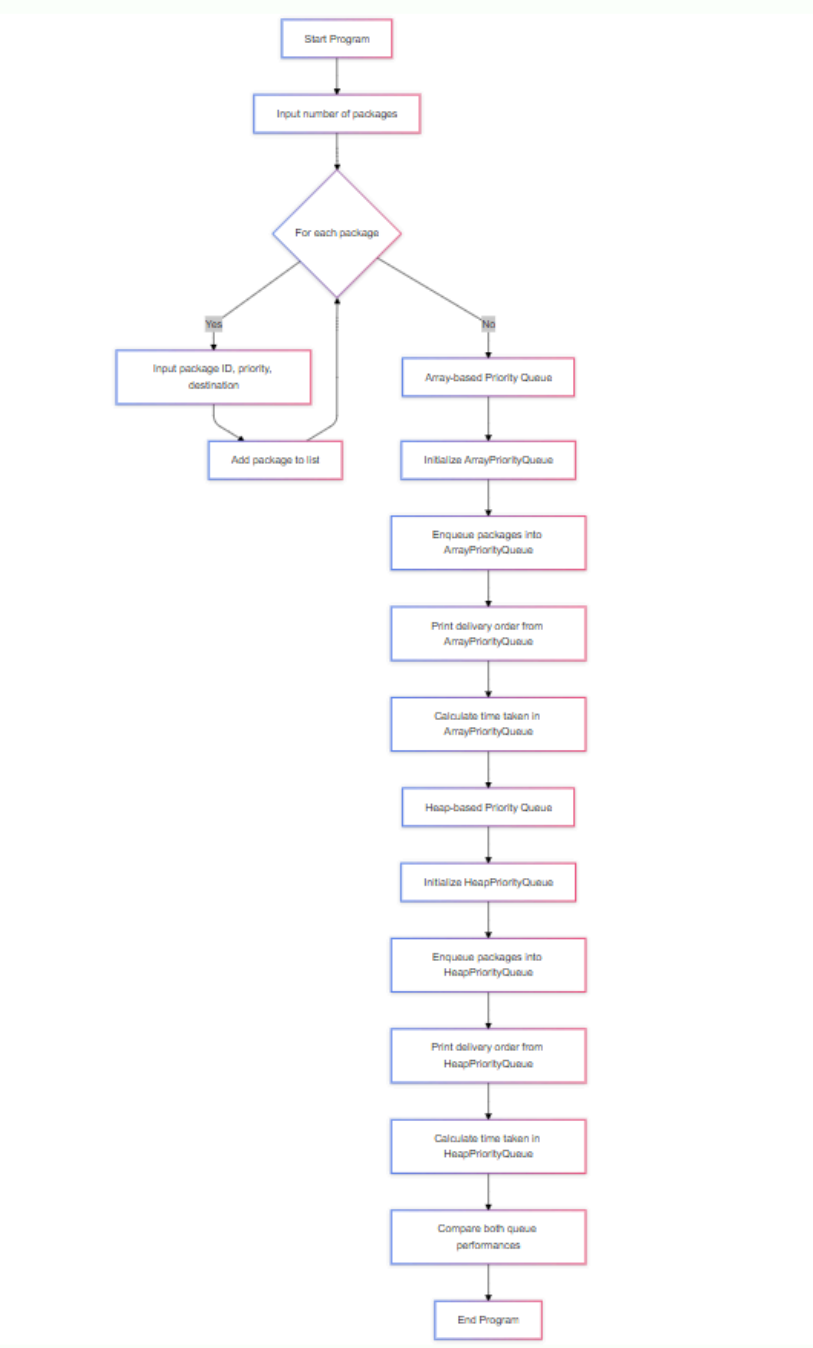
updateVariable(String var, int value)

- Updates the value of a specific variable in the HashMap.

interact(Scanner input)

- Provides an interactive interface for updating variables and displaying results.

Flowchart:



PROGRAM #3

CODE LINK

A delivery company operates in a small town with 5 main districts. Each district has multiple delivery points, and there are various routes between districts with different travel times.

Data Structures used:

Structure 1: Array-based Priority Queue
ArrayList: This is the underlying data structure used to store the packages.
Sorting: To maintain priority order, the entire ArrayList is sorted after each insertion. This is the less efficient approach, especially for large datasets.

Structure 2: Heap-based Priority Queue:
PriorityQueue: This is a built-in Java class that implements a priority queue using a heap data structure. It ensures efficient insertion and removal of elements based on their priority.
Min-Heap: The PriorityQueue is configured as a min-heap, where the element with the lowest priority is at the root. This allows for quick access to the highest priority package.

Structure 3: Graph

Algorithms Implemented:

Route Finding

Step 1: Start Program

Step 2: Execute Dijkstra's Algorithm

- The program initializes distances and processes the graph
- Iteratively evaluates neighbors and updates shortest paths

Step 3: Execute BFS Algorithm

- The program initializes a queue and processes the graph

Step 4: Compare Results

- The program evaluates both algorithms.

- Performance Analysis: BFS Algorithm is more efficient by 3 operations.

Step 5: End Program

Priority Queue
Step 1: Start Program
Step 2: Enter Package Details
Step 3: Array-Based Priority Queue
- Packages are sorted by priority after each insertion

Step 4: Heap-Based Priority Queue
- Packages are inserted into a heap structure

Step 5: Compare Results
- The program compares the performance of both approaches.
- Comparison: The array-based priority queue performs sorting after each enqueue, which is slower. The heap-based approach is more efficient and faster for larger datasets.

Key Functions:

Route Finding

dijkstra(String start, String destination)
- This method implements Dijkstra's algorithm to find the shortest path from the starting district (start) to the destination district (destination) in the graph.

bfs(String start, String destination)
- This method implements the Breadth-First Search (BFS) algorithm to find a path from the starting district (start) to the destination district (destination) in the graph.

reconstructPath(String destination, Map<String, String> previousNodes)
- This method reconstructs the path from the destination district (destination) back to the starting point using the information stored in the previousNodes map.

calculateTotalTime(List<String> path)
- This method calculates the total travel time for a given path (path).

compareRouteFinding(String start, String destination)
- This method compares the performance of Dijkstra's and BFS algorithms for finding the route between two districts.

printResult(Result result)
- This method helps format and print the details stored in a Result object for a specific algorithm.

Node class

- This is a simple class used internally by Dijkstra's algorithm.

Priority Queue

Package

- This class represents a single package with attributes like id, priority, and destination. It has a constructor to initialize these attributes.

ArrayPriorityQueue

- This class implements a priority queue using an underlying ArrayList.

enqueue(Package p)

- This method adds a package to the queue. It first adds the package to the list and then sorts the entire list by priority using a lambda expression.

printDeliveryOrder()

- This method iterates through the sorted list and prints information about each package (ID, priority, destination), along with the total number of operations performed.

HeapPriorityQueue

- This class implements a priority queue using the built-in java.util.PriorityQueue class. It utilizes a min-heap where the element with the lowest priority is at the root.

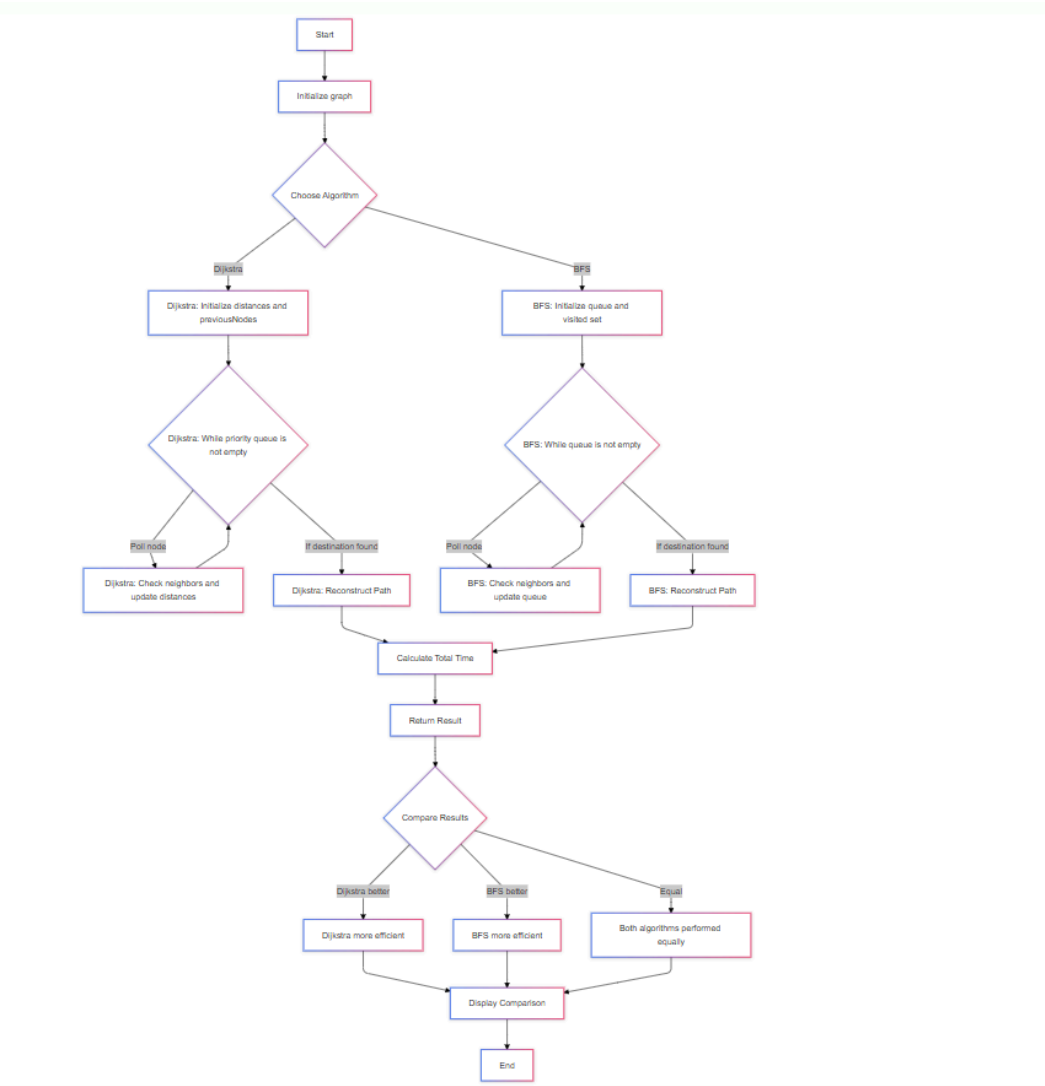
enqueue(Package p)

- This method adds a package to the priority queue using the offer method. This is a more efficient operation compared to sorting the entire list.

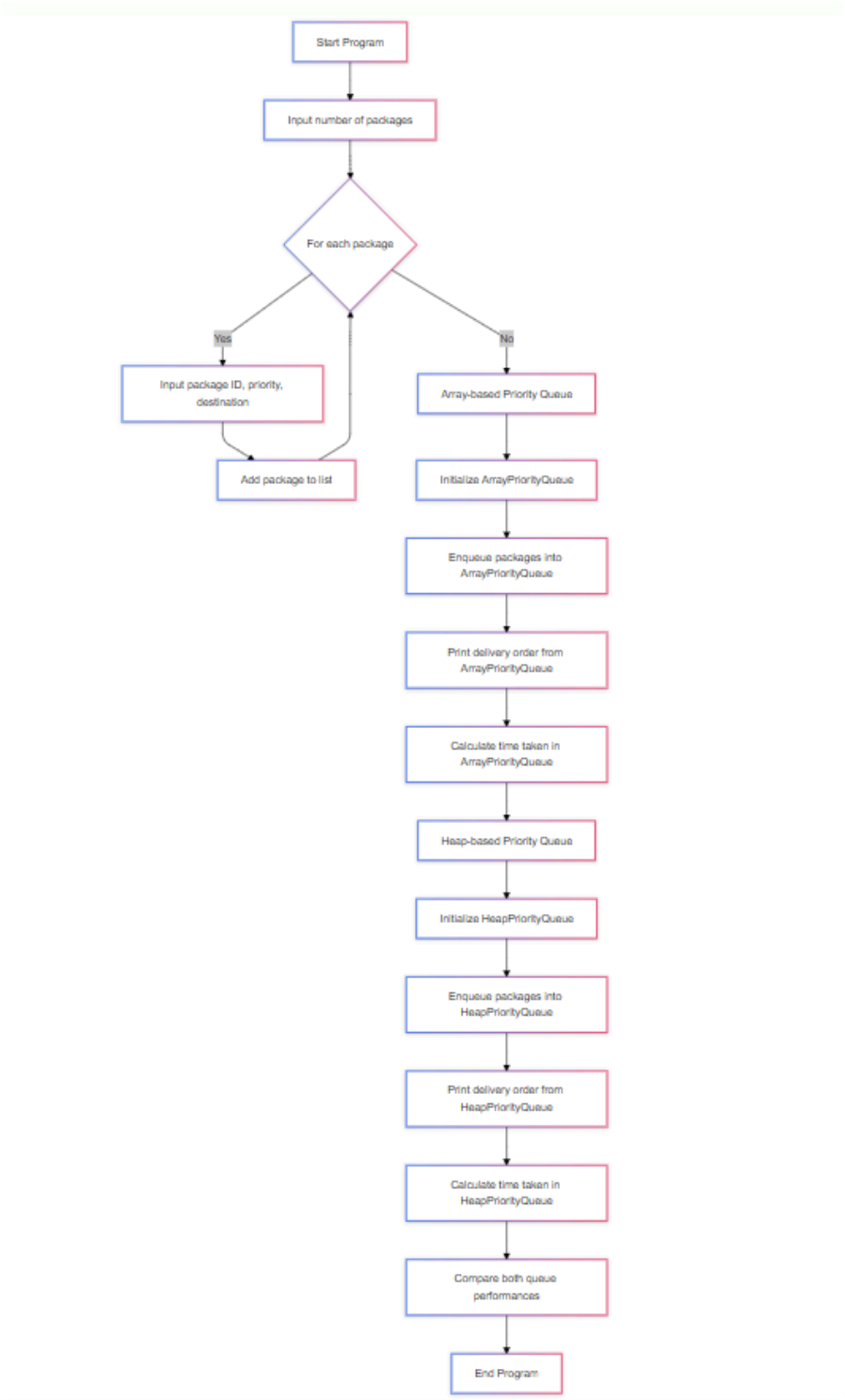
printDeliveryOrder()

- This method iterates through the elements of the priority queue using poll (which removes and returns the element with the lowest priority) and prints information about each package along with the total number of operations performed.

Flowchart 1:


Start
Initialize graph
Choose Algorithm
Dijkstra
BFS
Dijkstra: Initialize distances and previousNodes
BFS: Initialize queue and visited set
Dijkstra: While priority queue is not empty
BFS: While queue is not empty
Poll node
If destination found
Poll node
If destination found
Dijkstra: Check neighbors and update distances
Dijkstra: Reconstruct Path
BFS: Check neighbors and update queue
BFS: Reconstruct Path
Calculate Total Time
Return Result
Compare Results
Dijkstra better
BFS better
Equal
Dijkstra more efficient
BFS more efficient
Both algorithms performed equally
Display Comparison
End

Flowchart 2:


Start Program
Input number of packages
For each package
Yes
No
Input package ID, priority, destination
Add package to list
Array-based Priority Queue
Initialize ArrayPriorityQueue
Enqueue packages into ArrayPriorityQueue
Print delivery order from ArrayPriorityQueue
Calculate time taken in ArrayPriorityQueue
Heap-based Priority Queue
Initialize HeapPriorityQueue
Enqueue packages into HeapPriorityQueue
Print delivery order from HeapPriorityQueue
Calculate time taken in HeapPriorityQueue
Compare both queue performances
End Program

CONCLUSION

This project showcases the practical application of data structures and algorithms in solving real-world problems, emphasizing efficiency and scalability.

Program 1: Simple Menu

Using foundational data structures like arrays and loops, this program demonstrates how control structures can efficiently manage common tasks such as leap year validation, height comparison, and Fibonacci series generation. While simple, it lays the groundwork for user-interactive systems.

Program 2: Math Expression Calculator By implementing a binary expression tree, this program illustrates the power of recursive algorithms for parsing and evaluating arithmetic expressions. It dynamically updates variables and outputs prefix and postfix notations, bridging theoretical concepts with real-world applications like calculators and compilers.

Program 3: Delivery Route Optimization

This program solves a complex logistics problem using graph-based algorithms and priority queues. Dijkstra's algorithm efficiently determines the shortest path for weighted graphs, while BFS offers a simpler alternative for unweighted cases. The priority queue implementations (array-based and heap-based) handle package deliveries by urgency, with heap-based queues proving more efficient for larger datasets.

Key Takeaways:

- Algorithm Selection: Dijkstra's algorithm and heap-based priority queues stood out for their scalability and efficiency in real-world scenarios.
- Data Structure Utility: Binary trees, graphs, and priority queues were crucial in solving diverse problems, highlighting the importance of aligning structure with functionality.
- Real-World Relevance: Each program simulates practical scenarios where optimizing algorithms directly impacts performance, from user-facing tasks to logistics planning.

Future Enhancements:

- Introduce graphical interfaces for enhanced interactivity.

- Scale Program 3 to handle larger maps and datasets.
- Expand analysis to include additional algorithms for route finding and priority management.
- Expand error handling on program 2

This project reinforces how thoughtful integration of data structures and algorithms can transform theoretical knowledge into impactful solutions, preparing us to tackle real-world challenges with confidence.